

CSS 430 Operating Systems

Program 2B: CPU Scheduling – Multi-Level Feedback Queue

1. Purpose

This assignment implements the multilevel feedback-queue (MFQ) scheduling algorithm on top of Linux and compares it with the pre-implemented round-robin (RR) scheduling algorithm, using `kill(pid, SIGCONT)` and `kill(pid, SIGSTOP)`, each resuming and suspending a given process with `pid`.

2. Code Structure

The 2B folder includes three `cpp` files:

File names	Descriptions
driver.cpp	This program includes <code>main()</code> that instantiates a Scheduler object (named <code>scheduler</code>); spawns a given number of child processes, each running “task” for a certain number of seconds; passes all their child PIDs into <code>scheduler.addProcess()</code>, starts the scheduler with an independent thread, and waits for all the child process termination. Two cycle of scheduler execution will be performed: one with round robin, (i.e., <code>scheduler->run_rr()</code>), the other with multi-level feedback queue, (i.e., <code>scheduler->run_mfg()</code>). No need to modify this program. An execution format is: <code>./driver taskA 10 taskB 2 taskC 6 taskD 12 taskE 1</code>
task.cpp	This program receives a task name and a number of seconds to run. It simply calculates dummy <code>pow()</code> functions until its CPU time reaches the given number of seconds. No need to modify this program.
scheduler.cpp	This is <code>scheduler.h</code>’s implementation file. As you can see its header file below: <pre>#define DEFAULT_QUANTUM 1000000 // 1 sec using namespace std; class Scheduler { public: Scheduler() { }; Scheduler(int quantum); bool addProcess(int pid); // push pid into the queue at level 0. void run_rr(); // run processes as a round-robin scheduler void run_mfq(); // run processes as a multi-level feedback queue scheduler private: int msec = DEFAULT_QUANTUM; // a default time quantum int nQuantums = 0; // # quantums spent so far std::queue<int> queue[3]; // a multi-level queue void schedulerSleep(); // let the scheduler wait for a given quatumn to run the current process };</pre> driver.cpp calls <code>addProcess(pid)</code> to enqueue this child process into <code>queue[0]</code>. Either <code>run_r()</code> or <code>run_mfq()</code> will be executed with an independent thread. The former is round robin while the latter is multi-level feedback queue. The <code>run_rr()</code> method was already implemented. Your work is to implement <code>run_mfg()</code>.

3. Linux System Calls

Program 2B uses only the `kill()` system call but call it in three different ways, each summarized in the following table.

Linux System Functions	Actions
<code>kill(pid, 0)</code>	Checks if a given <code>pid</code> is active. It returns 0 if <code>pid</code> is active. Otherwise, you should get rid of it from the corresponding queue.
<code>kill(pid, SIGCONT)</code>	Resumes a process with <code>pid</code> .
<code>kill(pid, SIGSTOP);</code>	Suspends a process with <code>pid</code> .

Look at `scheduler_template.cpp`. In the `run_rr()` function, you can find the code snippet:

```
if ( kill( current, 0 ) == 0 ) { // current process is alive
    cerr << "\nscheduler: resumed " << current << endl;
    kill( current, SIGCONT ); // run it for a next quantum
    schedulerSleep( );
}
```

CSS 430 Operating Systems

Program 2B: CPU Scheduling – Multi-Level Feedback Queue

```
kill( current, SIGSTOP );
}
```

Make sure to resume only an active process.

4. Statement of Work

Task1: MFQ Implementation. Implement an MFQ algorithm in `rscheduler_template.cpp`'s `run_mfq()` as RR has been implemented in `run_rr()`. The generic algorithm is described in the textbook. Your multilevel feedback-queue scheduler must have the following specification:

1. It has three queues, numbered from 0 to 2, (i.e., `queue[0]`, `queue[1]`, and `queue[2]`). Level 0 is the highest while level 2 is the lowest. (Already implemented.)
2. A pid is always enqueued into `queue[0]` in `addProcess( pid )`. (Already implemented.)
3. Your scheduler first executes all processes in queue 0. The queue 0's time quantum is 1 second. (Already implemented.)
4. If a process in `queue[0]` does not complete its execution for `queue[0]`'s time slice, (i.e., 1 sec), the scheduler moves the corresponding pid to `queue[1]`.
5. If `queue[0]` is empty, it will execute processes in `queue[1]`. The `queue[1]`'s time quantum is twice larger than `queue[0]`'s, (i.e. 2 seconds). However, in order to respond to new processes in `queue[0]`, your scheduler should execute a process in `queue[1]` for 1 sec and then check if `queue[0]` has new pids. If so, it will execute all processes in `queue[0]` first, and thereafter resume the execution of the same process in `queue[1]` for another 1 second.
6. If a process in `queue[1]` does not complete its execution for `queue[1]`'s time quantum, (i.e., 2 seconds), the scheduler then moves its pid to `queue[2]`.
7. If both `queue[0]` and `queue[1]` is empty, it can execute processes in `queue[2]`. The `queue[2]`'s time quantum is a double of `queue[1]`, (i.e., 4 seconds). However, in order to respond to processes with higher priority in `queue[0]` and `queue[1]`, your scheduler should execute a process in `queue[2]` for every 1 second and then check if `queue[0]` and `queue[1]` have new pids. The rest of the behavior is the same as that for `queue[1]`.
8. If a process in `queue[2]` does not complete its execution for queue 2's time slice, (i.e., 4 seconds), the scheduler puts it back to the tail of `queue[2]`. (This is different from the textbook example that executes processes in queue 2 with FCFS.)

Focus on `run_mfq()`. You may use similar logic from `run_rr()`.

Task2: Verification. Run `driver.cpp` as follows.

```
[css430@cssmpi1 2B]$ ./compile.sh
[css430@cssmpi1 2B]$ ls
compile.sh driver.cpp scheduler.cpp scheduler.h scheduler.o scheduler_template.cpp
task task.cpp
[css430@cssmpi1 2B]$ ./driver taskA 10 taskB 2 taskC 6 taskD 12 taskE 1
driver: task[taskA] runs for 10seconds.
driver: task[taskB] runs for 2seconds.
driver: task[taskC] runs for 6seconds.
driver: task[taskD] runs for 12seconds.
driver: task[taskE] runs for 1seconds.
session 0:round robin scheduler -----
parent = 9068
scheduler (round robin): started

scheduler: resumed 9069
driver: child = 9069
task[taskA] with pid=9069 runs for 10 seconds
taskA
taskA
scheduler: completed 1 quantums

scheduler: resumed 9070
driver: child = 9070
task[taskB] with pid=9070 runs for 2 seconds
taskB
taskB
scheduler: completed 2 quantums

scheduler: resumed 9071
driver: child = 9071
```

CSS 430 Operating Systems
Program 2B: CPU Scheduling – Multi-Level Feedback Queue

task[taskC] with pid=9071 runs for 6 seconds
taskC
taskC
scheduler: completed 3 quantum

scheduler: resumed 9072
driver: child = 9072
task[taskD] with pid=9072 runs for 12 seconds
taskD
taskD
scheduler: completed 4 quantum

scheduler: resumed 9073
driver: child = 9073
task[taskE] with pid=9073 runs for 1 seconds
taskE

Task[taskE]: cpu time used: 1.000050 seconds (answer = 80.871546) count = 14791
scheduler: completed 5 quantum
scheduler: confirmed 9073's termination

scheduler: resumed 9069
taskA
taskA
scheduler: completed 6 quantum

scheduler: resumed 9070

Task[taskB]: cpu time used: 2.000164 seconds (answer = 80.871546) count = 29136
scheduler: completed 7 quantum
scheduler: confirmed 9070's termination

scheduler: resumed 9071
taskC
taskC
scheduler: completed 8 quantum

scheduler: resumed 9072
taskD
taskD
scheduler: completed 9 quantum

scheduler: resumed 9069
taskA
taskA
scheduler: completed 10 quantum

scheduler: resumed 9071
taskC
taskC
scheduler: completed 11 quantum

scheduler: resumed 9072
taskD
taskD
scheduler: completed 12 quantum

scheduler: resumed 9069
taskA
taskA
scheduler: completed 13 quantum

scheduler: resumed 9071

Task[taskC]: cpu time used: 6.000485 seconds (answer = 80.871546) count = 87409
scheduler: completed 14 quantum

CSS 430 Operating Systems
Program 2B: CPU Scheduling – Multi-Level Feedback Queue

```
scheduler: confirmed 9071's termination

scheduler: resumed 9072
taskD
taskD
scheduler: completed 15 quantums

scheduler: resumed 9069
taskA
taskA
scheduler: completed 16 quantums

scheduler: resumed 9072
taskD
taskD
scheduler: completed 17 quantums

scheduler: resumed 9069

Task[taskA]: cpu time used: 10.000437 seconds (answer = 80.871546) count = 146637
scheduler: completed 18 quantums
scheduler: confirmed 9069's termination

scheduler: resumed 9072
taskD
taskD
scheduler: completed 19 quantums

scheduler: resumed 9072

Task[taskD]: cpu time used: 12.000501 seconds (answer = 80.871546) count = 176037
driver: All tasks are done
scheduler: completed 20 quantums
scheduler: confirmed 9072's termination
scheduler: has no more process to run
driver: confirms the scheduler thread termination
session 1:multilevel feedback queue scheduler -----
-----
parent = 9068
scheduler (multilevel feedback queue): started

scheduler: resumed 9077
driver: child = 9077
task[taskA] with pid=9077 runs for 10 seconds
taskA
scheduler: completed 1 quantums

scheduler: resumed 9078
driver: child = 9078
task[taskB] with pid=9078 runs for 2 seconds
taskB
scheduler: completed 2 quantums

scheduler: resumed 9079
driver: child = 9079
task[taskC] with pid=9079 runs for 6 seconds
taskC
scheduler: completed 3 quantums

scheduler: resumed 9080
driver: child = 9080
task[taskD] with pid=9080 runs for 12 seconds
taskD
scheduler: completed 4 quantums

scheduler: resumed 9081
```

CSS 430 Operating Systems
Program 2B: CPU Scheduling – Multi-Level Feedback Queue

```
driver: child = 9081
task[taskE] with pid=9081 runs for 1 seconds
taskE
scheduler: completed 5 quantums

scheduler: resumed 9077
taskA
scheduler: completed 6 quantums

scheduler: resumed 9077
taskA
scheduler: completed 7 quantums

scheduler: resumed 9078
taskB
scheduler: completed 8 quantums

scheduler: resumed 9078

Task[taskB]: cpu time used: 2.000138 seconds (answer = 80.871546) count = 28970
scheduler: completed 9 quantums
scheduler: confirmed 9078's termination

scheduler: resumed 9079
taskC
scheduler: completed 10 quantums

scheduler: resumed 9079
taskC
scheduler: completed 11 quantums

scheduler: resumed 9080
taskD
scheduler: completed 12 quantums

scheduler: resumed 9080
taskD
scheduler: completed 13 quantums

scheduler: resumed 9081

Task[taskE]: cpu time used: 1.000158 seconds (answer = 80.871546) count = 14112
scheduler: completed 14 quantums
scheduler: confirmed 9081's termination

scheduler: resumed 9077
taskA
scheduler: completed 15 quantums

scheduler: resumed 9077
taskA
scheduler: completed 16 quantums

scheduler: resumed 9077
taskA
scheduler: completed 17 quantums

scheduler: resumed 9077
taskA
scheduler: completed 18 quantums

scheduler: resumed 9079
taskC
scheduler: completed 19 quantums

scheduler: resumed 9079
```

CSS 430 Operating Systems
Program 2B: CPU Scheduling – Multi-Level Feedback Queue

```
taskC
scheduler: completed 20 quantaums

scheduler: resumed 9079
taskC
scheduler: completed 21 quantaums

scheduler: resumed 9079

Task[taskC]: cpu time used: 6.000497 seconds (answer = 80.871546) count = 87177
scheduler: completed 22 quantaums
scheduler: confirmed 9079's termination

scheduler: resumed 9080
taskD
scheduler: completed 23 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 24 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 25 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 26 quantaums

scheduler: resumed 9077
taskA
scheduler: completed 27 quantaums

scheduler: resumed 9077
taskA
scheduler: completed 28 quantaums

scheduler: resumed 9077
taskA
scheduler: completed 29 quantaums

scheduler: resumed 9077

Task[taskA]: cpu time used: 10.000583 seconds (answer = 80.871546) count = 145677
scheduler: completed 30 quantaums
scheduler: confirmed 9077's termination

scheduler: resumed 9080
taskD
scheduler: completed 31 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 32 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 33 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 34 quantaums

scheduler: resumed 9080
taskD
scheduler: completed 35 quantaums
```

CSS 430 Operating Systems

Program 2B: CPU Scheduling – Multi-Level Feedback Queue

scheduler: resumed 9080

```
Task[taskD]: cpu time used: 12.000716 seconds (answer = 80.871546) count = 175037
driver: All tasks are done
scheduler: completed 36 quantaums
scheduler: confirmed 9080's termination
scheduler: has no more process to run
driver: confirms the scheduler thread termination
[css430@cssmpi1 2B]$
```

In the above example, `run_rr()` needed 20 quantaums in total. As each quantum is 2 seconds, it spent 40 seconds. On the other hand, `run_mfq()` needed 36 quantaums, which means 36 seconds. So, MFQ's total time is a little shorter than RR.

Task3: TAT Calculation. For each of tasks A-E, calculate its TAT, a time (in seconds) from its submission to termination. Please assume that all tasks were submitted at quantum 0. Recall that `run_rr()`'s quantum is 2 seconds while `run_mfq()`'s quantum is 1 second.

5. What to Turn in

	Materials	Points
1	Task 1: <code>run_mfq()</code> . a. Code organization: +2pts • Well organized: 2pts, • Poor comments or bad organization: 1pt, or • No comments and horrible code: 0pts b. Correctness: +8pts • <code>queue[0]</code>, <code>[1]</code>, and <code>[2]</code> guarantee 1 slice, 2 slices, and 4 slices of 1 second for running each process, respectively: +3pts • At each slice of 1 second, <code>run_mfq()</code> checks if any processes exist in higher-priority queues: +2pts, and • If the current process used up the slices of the current queue, it will go to the next lower queue or stay in the lowest queue: +3pts	10
2	Task 2: an execution snapshot in txt, jpg, png, or pdf a. <code>run_mfq()</code>'s behavior: 3pts • Longer processes go down from <code>queue[0]</code> to <code>queue[2]</code>: 3pts • Little difference between shorter and longer processes: 2pts, or • No results: 0pts b. <code>run_mfq()</code>'s execution performance: • <code>run_mfq()</code> performs better than <code>run_rr()</code>: 3pts • <code>run_mfq()</code> does not outperform <code>run_rr()</code>: 2pts • No results: 0pts	6
3	Task 3: TAT calculation • <code>run_mfq()</code>'s TAT is generally shorter than <code>run_rr()</code>: 4pts • <code>run_mfq()</code> and <code>run_rr()</code> have no difference in TAT: 2pts • No results: 0pts	4

Total 20pts.